

Pose-IQ — מסמך סטטוס

מסמך פנימי להכנה לפגישת חיתוך מצב עם המנחה · עודכן ביוני 2026

Pose-IQ היא מערכת אימון כושר חכמה בזמן אמת. המצלמה מצלמת את המתאמן, המערכת מזהה את שלד הגוף, מודדת זוויות במפרקים, מזהה את שלב התרגיל וסופרת חזרות, נותנת משוב מיידי על טעויות יציבה (ויזואלי + קולי), שומרת היסטוריית אימונים ומפיקה המלצות מותאמות אישית. בנוסף נבנתה שכבת **למידת מכונה** שלומדת מתוך סרטוני הדגמה, ולאחרונה נוספה **שכבת מסד נתונים** כהכנה למעבר למוצר רב-משתמשים.

תוכן העניינים

- סקירה כללית וסטאק טכנולוגי
- מבנה התיקיות והקבצים — תפקיד כל רכיב
- זרימת המערכת (ארכיטקטורה)
- שכבת למידת המכונה (ML)
- שכבת מסד הנתונים (DB)
- מה כבר עובד והושג
- התלבטויות להמשך
- מה עוד חסר — צעדים הבאים

1. סקירה כללית וסטאק טכנולוגי

כיום המערכת רצה כאפליקציית דסקטופ מקומית בפייתון: נפתח חלון `OpenCV` שמציג את וידאו המצלמה עם שלד מצויר, מד-זוויות, מונה חזרות, רשימת מוכנות ותיבות שגיאה.

תחום	טכנולוגיה
שפה	Python 3.12
ראייה ממוחשבת / מצלמה	OpenCV
זיהוי פוז (שלד)	MediaPipe Pose Landmarker (33 נקודות גוף)
חישוב מספרי	NumPy
למידת מכונה	joblib , pandas , scikit-learn
משוב קולי	pyttsx3 (טקסט-לדיבור)
מסד נתונים חדש	SQLite + SQLAlchemy (פיתוח) / PostgreSQL (ייצור)

חמישה תרגילים מוגדרים כיום: **סקוואט, פלאנק, לאנג', בייספ קרל, לחיצת כתפיים**. המערכת עוקבת אחר **8 זוויות גוף**: ברך ימין/שמאל, מרפק ימין/שמאל, זווית זרוע-גוף ימין/שמאל, גב (spine) ופיסוק רגליים (legs_spread).

2. מבנה התיקיות והקבצים — תפקיד כל רכיב

כל הקוד תחת `core/`, מחולק לתת-חבילות לפי אחריות. הנתונים תחת `data/`.

`/core/detection` — ראייה ממוחשבת וזיהוי

קובץ	תפקיד
<code>camera_stream.py</code>	עטיפה למצלמה דרך OpenCV: פתיחה, קריאת פריימים, חישוב FPS.
<code>pose_detector.py</code>	טוען את מודל ה-MediaPipe (מוריד אוטומטית בפעם הראשונה), מקבל פריים ומחזיר את 33 נקודות השלד עם קואורדינטות ורמת נראות (visibility).
<code>angle_calculator.py</code>	הופך נקודות שלד ל-8 זוויות גוף, באמצעות חישוב זווית תלת-ממדית ($\arccos$) של מכפלה סקלרית) בין שלשות מפרקים.

/core/exercise — מודל התרגיל ולוגיקת המצב

קובץ	תפקיד
<code>exercise_model.py</code>	טוען את הגדרות התרגילים מ- <code>exercises.json</code> למבני נתונים (תרגיל ← שלבים ← טווחי-זווית). כולל <code>match_phase</code> שמנחש את השלב הקרוב ביותר (לשחזור אוטומטי).
<code>exercise_state_machine.py</code>	מכונת המצבים של חזרה: בודקת מוכנות לפני התחלה, מזהה מעברים בין שלבים (לאחר 5 פריימים יציבים), סופרת חזרות, ומבצעת <i>שחזור אוטומטי</i> אם המתאמן יצא מהפריים ~3 שניות וחזר.
<code>posture_rules.py</code>	מנוע החוקים: משווה זוויות בפועל מול הטווחים המותרים, מייצר התראות יציבה עם חומרה והודעת תיקון. מתאים את הסף לפי רמת הכושר ולפי מגבלות פיזיות אישיות (מרחיב טווח למפרק פגיע), עם השהיית-זמן (debounce) למניעת התראות שווא.

/core/user — פרופיל, היסטוריה ותכנון

קובץ	תפקיד
<code>user_profile.py</code>	פרופיל המשתמש: שם, רמת כושר, מגבלות, תרגילים מועדפים, סגנון מאמן, ומזהה יציב (<code>user_id</code>). שומר ל-JSON.
<code>onboarding.py</code>	אשף הקמת פרופיל אינטראקטיבי במסך הראשון (שאלות על שם/רמה/מגבלות/העדפות).
<code>workout_history.py</code>	שמירת סשני אימון (חזרות, מפרקים שגויים, ציון פורם, משך), וחישוב מדדי התקדמות: ממוצע אחרון, מגמה (משתפר/יורד/יציב), ומפרקים חלשים.
<code>workout_plan.py</code>	מנוע ההמלצות: מדרג תרגילים לסשן, מתאים יעד חזרות לפי הרמה והמגמה, וממליץ על תרגילים מתקנים לאזורים החלשים. כולל מיפוי מגבלה ← תרגילים שעלולים להזיק.

/core/coaching — משוב קולי

קובץ	תפקיד
<code>coach_messages.py</code>	מאגר משפטים ב-3 סגנונות מאמן (קשוח / רגוע / מוטיבטור), לכל אירוע: התחלה, מעבר שלב, סיום חזרה, שגיאת מפרק, עידוד, סיום סשן.
<code>voice_coach.py</code>	הקראת המשובים בקול (TTS) ב-thread רקע, עם cooldown לכל סוג הודעה כדי לא להציף את המתאמן.

/core/ml — שכבת למידת המכונה

קובץ	תפקיד
<code>data_collector.py</code>	אוסף "תמונת מצב" של הזוויות בכל פריים במהלך אימון חי, ושומר ל-CSV (סכמה אחידה). מקור-אמת יחיד לשורת נתונים.
<code>video_extractor.py</code>	חילוץ לא-מקוון: עובר על סרטוני הדגמה ומפיק את אותה סכמת CSV, כך שסרטונים ואימונים חיים מזינים צינור-אימון אחד.
<code>features.py</code>	הנדסת מאפיינים: הופך חלון של ~25 פריימים לוקטור מאפיינים קבוע (5 אגרגציות לכל מפרק = 40 מאפיינים). אותו קוד משמש באימון ובחיזוי החי — כדי שלא ייווצר פער.
<code>trainer.py</code>	מאמן את "ראשי" המודל: זיהוי תרגיל, זיהוי שלב, ואיכות-פורם. שומר את המודלים ומדווח דיוק.
<code>store.py</code>	שמירה/טעינה של מודלים מאומנים (מקור-אמת יחיד לנתיבי הקבצים + רישום).
<code>predictor.py</code>	חיזוי בזמן אמת: מחזיק חלון מתגלגל של פריימים, מריץ את המודל ומחזיר תחזית + ביטחון. מוכן לחיבור לאפליקציה.

/core/db — שכבת מסד הנתונים חדש

קובץ	תפקיד
database.py	חיבור ה-DB: engine + sessions. אגנוסטי למסד — פיתוח על SQLite, ייצור על PostgreSQL, החלפה דרך משתנה-סביבה אחד בלבד.
models.py	טבלאות ה-ORM: משתמשים ← סשנים ← חזרות ← שגיאות-מפרק. כולל עמודת weight_kg חדשה (לקראת המלצות משקל).
repositories.py	שכבת גישה לנתונים: שמירת סשן, שליפת היסטוריה, וחישוב "מפרקים חלשים" כשאילתת GROUP BY יחידה.
migrate_json.py	הגירה חד-פעמית של ה-JSON הקיים אל מסד הנתונים (אידמפוטנטית — לא משכפלת).

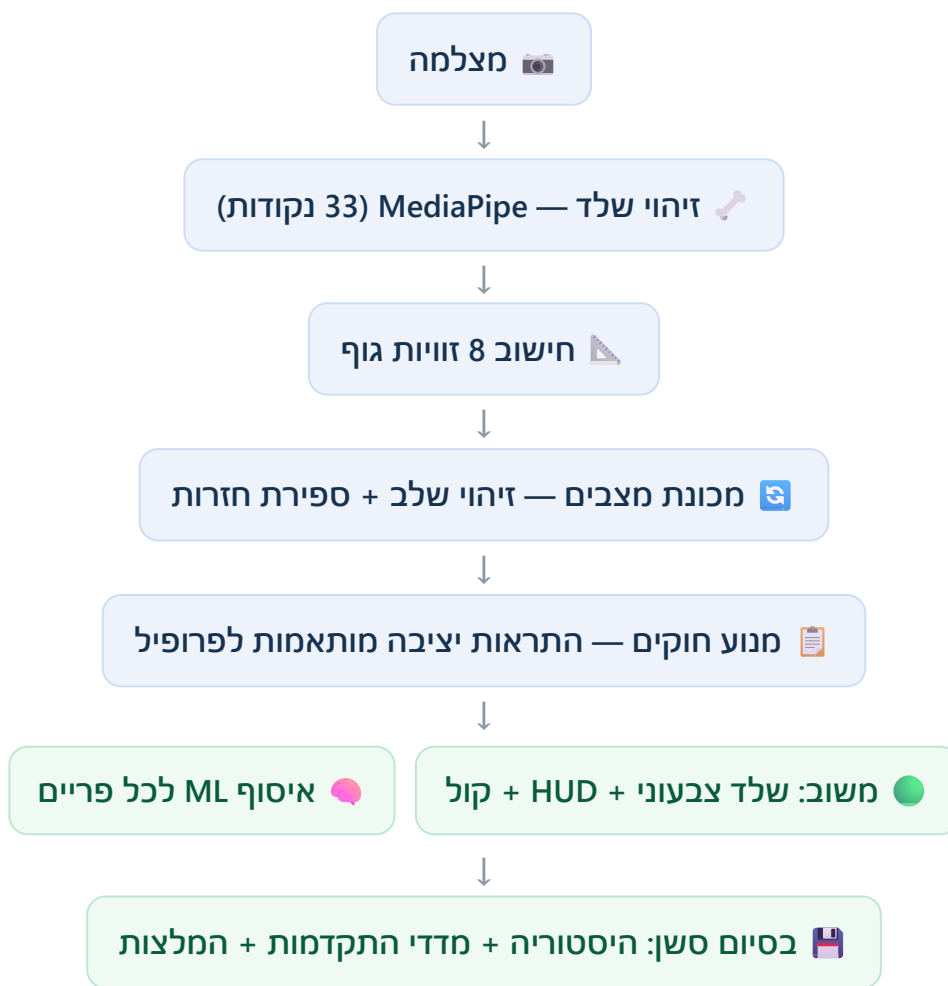
קבצים ראשיים ותיקיית הנתונים

נתיב	תפקיד
core/pipeline.py	המונצח (orchestrator): מחבר את כל החלקים — מצלמה ← זיהוי ← זוויות ← מכונת מצבים ← חוקים ← משוב ויזואלי/קולי ← איסוף ML ← היסטוריה. מכאן מריצים את האפליקציה.
data/exercises.json	הגדרת כל התרגילים: שלבים, טווחי-זווית, הודעות תיקון, אילוצים גלובליים. הלב של המערכת — דאטה, לא קוד.
/ data/user_profile.json workout_history.json	אחסון מקומי קיים (יוחלף בהדרגה במסד הנתונים).
data/videos/	סרטוני הדגמה לאימון ה-ML (~191MB; מחוץ ל-git).
, data/models/ , data/video_data.csv data/poseiq.db	תוצרים שנוצרים אוטומטית (מודלים, דאטה מחולץ, מסד מקומי).
requirements.txt	רשימת התלויות (נוצרה כעת).

3. זרימת המערכת (ארכיטקטורה)

בכל פריים מהמצלמה רצה השרשרת הבאה:

1. **מצלמה** (camera_stream) → פריים.
2. **זיהוי פוז** (pose_detector) → 33 נקודות שלד.
3. **חישוב זוויות** (angle_calculator) → 8 זוויות גוף.
4. **מכונת מצבים** (exercise_state_machine) → בודקת מוכנות/מעבר שלב, סופרת חזרות.
5. **מנוע חוקים** (posture_rules) → רשימת התראות יציבה (מפרק, חומרה, הודעת תיקון), מותאמת לפרופיל.
6. **משוב:** ציור שלד צבעוני + HUD (pipeline) ומשוב קולי (voice_coach).
7. **איסוף ML** (data_collector) → שומר את הפריים לאימון עתידי.
8. בסיום סשן → **היסטוריה** (workout_history) ו**המלצות** (workout_plan).



4. שכבת למידת המכונה (ML)

הרעיון: מהזוויות הגולמיות לומדים דפוסים. הצינור: **איסוף** → **חילוץ מסרטונים** → **מאפיינים** → **אימון** → **חיזוי**. מאמנים שלושה "ראשים" עצמאיים:

ראש	מה הוא עושה	דיוק נוכחי
זיהוי תרגיל	מזהה איזה תרגיל מבוצע	~98%
זיהוי שלב	מזהה את שלב התנועה (עמידה/ירידה/דחיפה...)	~90%
איכות-פורם ניסיוני	מסמן סטייה מפורם תקין	ראה הערה ↓

המודלים הספציפיים — משפחת ה"יער" (Forest)

שני המודלים שבשימוש שייכים לאותה משפחה — **אנסמבל של עצי החלטה** (decision trees), ומכאן השם "יער":

1 Random Forest (יער אקראי) — לזיהוי תרגיל ושלב.

מה זה: מאות עצי החלטה, כל אחד מאומן על תת-קבוצה אקראית של הנתונים והמאפיינים; כל עץ "מצביע" והרוב מנצח. האקראיות מקטינה התאמת-יתר (overfitting).
למה דווקא הוא:

עובד מצוין על דאטה טבלאי **קטן** (יש לנו ~200 חלונות) — לא דורש כמויות עתק. לא דורש נירמול מאפיינים; תופס קשרים לא-ליניאריים בין 40 המאפיינים. מחזיר **הסתברויות** → רמת ביטחון לתצוגה; מהיר בחיזוי → מתאים לזמן אמת.

2 Isolation Forest (יער בידוד) — לאיכות-פורם (זיהוי חריגות).

מה זה: גם יער של עצים, אך בנוי כדי **לבדד חריגות**: פיצולים אקראיים מבודדים דגימה חריגה (פורם שגוי) במעט צעדים — מסלול קצר ← חריגה; פורם תקין דורש יותר פיצולים.
למה דווקא הוא: מושלם למצב של "**רק דוגמאות נכונות**" — לומד את מעטפת ה"נכון" ומסמן סטיות, בלי אף דוגמת "רע". אותה משפחה, אותם יתרונות: מהיר, ללא נירמול, ומחזיר ציון חריגות.

נקודה חשובה להצגה: אין עדיין דוגמאות של פורם "שגוי"

כרגע יש רק סרטוני ביצוע **נכון**. מסווג טוב/רע רגיל דורש שתי מחלקות, ולכן המערכת נופלת אוטומטית ל-**זיהוי חריגות (one-class)**: לומדים את מעטפת ה"נכון" לכל תרגיל, וכל סטייה ממנה מסומנת כחשודה. כך מתקבל סיגנל-פורם גם בלי דוגמאות שגויות. כשנוסיף סרטוני פורם שגוי — נשדרג למסווג בינארי אמיתי, ללא שינוי בצד החי.

בנוסף: חלק מסרטוני ההדגמה מהאינטרנט כוללים קטעי הסבר (מדריך עומד ומדבר) שמזהמים מעט את הדאטה — נטופל בעתיד ע"י חיתוך.

שכבת ה-ML **חोברה לאפליקציה החיה**: זיהוי תרגיל/שלב רצים חי, ואיכות-הפורם מוצגת כרגע רק במצב debug (מקש d) — בכוונה, עד שהדאטה תנוקה, כדי לא להציג אזהרות שווא.

האימון בוצע בספריית `scikit-learn` — עם `pandas` לעיבוד הנתונים ו-`joblib` לשמירת המודלים. אותו `features.py` מחשב את המאפיינים גם באימון וגם בחיזוי החי, כדי שלא ייווצר פער ביניהם.

למה צריך גם חוקים קשיחים וגם מודלי ML? (שאלה צפויה)

הם עושים עבודות **שונות ומשלימות**, לא מתחרות:

- **שני המודלים** מבצעים **זיהוי** (איזה תרגיל, איזה שלב) ו**סימון חריגה** ("הפורם נראה חריג") — אבל אינם אומרים **מה** שגוי ואיך לתקן.
- **החוקים** נותנים בדיוק את זה: משוב **ספציפי, מוסבר ופעיל** ("הגב מעל 30°, יישר אותו"), אכיפת **בטיחות** (אילוצים גלובליים), ו**בסיס שעובד מהיום הראשון** ללא דאטה ובלי טעויות-מודל.
- החוקים אפילו **מאמנים** את ה-ML: התיוג החלש (ציון הפורם) מגיע ממנוע החוקים.

בקיצור: החוקים הם השדרה הדטרמיניסטית והמוסברת (תיקון בזמן אמת + בטיחות + בסיס המלצות), וה-ML מוסיף יכולות שקשה לחוקים — זיהוי תרגיל/שלב חסין מתוך תנועה גולמית, וסיגנל-פורם הוליסטי שתופס דפוסים עדינים. **שילוב** חוקים + ML הוא הסטנדרט במערכות אמיתיות.

5. שכבת מסד הנתונים (DB)

היום הכול נשמר בקבצי JSON מקומיים, למשתמש יחיד. כדי לתמוך במשתמשים אמיתיים, היסטוריה לאורך זמן והמלצות — נדרש מסד נתונים אמיתי. נבנתה שכבה ראשונית:

בחירת מסד: PostgreSQL (יחסי) ולא Mongo — הנתונים יחסיים מובהקים (משתמשים ← סשנים ← חזרות ← שגיאות), וכל האנליטיקות הן אגרגציות SQL.

אגנוסטי למסד: מפתחים על SQLite מקומית, בייצור מצביעים על Postgres (Supabase/Neon, חינומי) ע"י שינוי משתנה-סביבה אחד — אין צורך להתקין Postgres עכשיו.

הוגרו נתוני הפרופיל הקיימים; שאילתת "מפרקים חלשים" אומתה כ- `GROUP BY` יחיד.

נוספה עמודת `weight_kg` לקראת המלצות משקל.

6. מה כבר עובד והושג

- זיהוי שלד בזמן אמת + חישוב 8 זוויות גוף.
- 5 תרגילים מוגדרים, עם זיהוי שלב, ספירת חזרות ושחזור אוטומטי מאיבוד מעקב.
- מנוע חוקי יציבה עם משוב מיידי, מותאם לרמת כושר ולמגבלות פיזיות.
- משוב ויזואלי (שלד צבעוני + HUD) ומשוב קולי ב-3 סגנונות.
- פרופיל משתמש, היסטוריית אימונים, מדדי התקדמות והמלצות תרגילים מותאמות.
- שכבת ML מלאה: זיהוי תרגיל (~98%) ושלב (~90%), + זיהוי חריגות לפורם, מחוברת לאפליקציה.
- שכבת מסד נתונים ראשונית (סכמה + הגירה + שאילתות), מוכנה ל-Postgres.

7. התלבטויות להמשך

א. React Native מול אפליקציית ווב (להחלטה עם המנחה)

היעד: שהמערכת תרוץ כאפליקציה למשתמשים אמיתיים. השאלה אם ללכת ל-**React Native** (מובייל) או לאפליקציית ווב.

אילוץ מכריע: זיהוי הפוז רץ על וידאו חי — חייב לרוץ **בצד הלקוח** (אי אפשר לשדר מצלמה גולמית לשרת בזמן אמת). לכן **צד-השרת זהה בשני המקרים** (API דק + DB), וההבדל הוא רק בלקוח.

ההשלכה ל-RN: צינור הזמן-אמת הנוכחי בפייתון *אינו עובר* כפי שהוא — הוא הופך ל"מפרט ייחוס", ויש לכתוב מחדש את לולאת הזמן-אמת ב-TypeScript על המכשיר. עם זאת, `exercises.json` והלוגיקה הדטרמיניסטית עוברים בקלות, וכל צד-השרת ממחזר את הפייתון הקיים.

שרת — FastAPI + PostgreSQL

חשבונות ואימות
היסטוריה + המלצות
אימון ML + חיזוי לפי חלון

↔
נתונים מסוכמים
בלבד · API

לקוח — RN (מובייל) או ווב

מצלמה + זיהוי פוז (על המכשיר)
זוויות + מכונת מצבים + חוקים
משוב בזמן אמת + שלד

צד-השרת זהה בשני המסלולים — ההחלטה RN/ווב משנה רק את הלקוח.

ב. המודל ללמידת המשתמש — ומה עד שהוא מוכן

מה הוא יחזה: את ההמלצה האישית לסשן הבא — בעיקר **המשקל/העומס** המתאים ורמת הקושי (יעד חזרות), על בסיס היסטוריית המשתמש (ציוני פורם אחרונים, מגמה, נפח, מפרקים חלשים).

איזה מודל ולמה: זו בעיית **רגרסיה על דאטה טבלאי** עם מאפיינים מהונדסים מההיסטוריה. הבחירה הטבעית: **עצי החלטה מועצמים (Gradient Boosting — XGBoost/LightGBM)**, או **Random Forest Regressor** מאותה משפחה כמו היום. למה:

- חזקים במיוחד לחיזוי טבלאי (סטנדרט בתעשייה).
 - **מוסברים** — אפשר להראות אילו גורמים השפיעו על ההמלצה (feature importance); חשוב לאמון המשתמש ולהגנה האקדמית.
 - עובדים עם כמות דאטה סבירה, בלי רשת נוירונים כבדה; ועקביים עם הסטאק הקיים.
- כמה דאטה נצטרך:** מודל אישי משמעותי דורש **היסטוריה לאורך זמן פר-משתמש** — בסדר גודל של **עשרות שנים למשתמש**, ולמודל חוצה-משתמשים **מאות-אלפי שנים** בסך הכול. בתחילת הדרך אין כמות כזו, ולכן מודל מאומן עדיין לא יכה חוקים פשוטים.

מה עושים עד אז — החוקים שכבר קיימים אצלנו:

- **דירוג תרגילים:** ניקוד — מועדף (+3), מכון לאזור חלש (+2), זקוק לתרגול אם ציון אחרון $70 > (+2)$, גיוון (+1); תרגיל שמתנגש עם מגבלה פיזית נפסל.
- **התאמת קושי:** יעד חזרות בסיסי לפי רמה (8/12/16); ציון $85 \leq$ ומגמה משתפרת $\rightarrow +3$ חזרות וטווחים מחמירים; ציון > 60 או מגמה יורדת $\rightarrow -3$ חזרות וטווחים מקלים.
- **המלצות מתקנות:** תרגילים שמחזקים את המפרקים החלשים (לפי ספירת שגיאות).
- **התאמת ספי-יציבה:** לפי רמת כושר ($1.3 \times / 1.0 / 0.85$) ולפי מגבלות (הרחבת טווח $1.4 \times$ והורדת חומרה).
- **המלצת משקל (חוק overload מתוכנן):** הושג יעד החזרות עם פורם טוב $\rightarrow$ להציע $+2.5$ ק"ג; הפורם נפגע / היעד לא הושג $\rightarrow$ לשמור או להפחית.

8. מה עוד חסר — צעדים הבאים

- **סרטוני פורם שגוי** → לשדרג את ראש האיכות למסווג בינארי אמיתי.
- **מעקב משקלים** → האפליקציה עדיין לא קולטת משקל; נדרש לפני המלצת משקל (העמודה כבר קיימת בסכמה).
- **צד-שרת (FastAPI)** → לעטוף את הלוגיקה ב-API לחשבונות, היסטוריה והמלצות.
- **הקמת מסד ייצור (Postgres מנוהל) + הגירה.**
- **צד-לקוח** → לאחר ההחלטה: RN (מובייל) או ווב; כולל מימוש זיהוי הפוז על המכשיר.
- **אנליטיקות מתקדמות** → נפח אימון, רצפים, זיהוי פלאטו לאורך זמן.
- **אחסון/דיפלווי** → העלאת השרת והמסד לענן כדי שמשתמשים אמיתיים יוכלו להשתמש.
- **פלאנק** → עדיין מדולג; להוסיף סרטונים ולכלול אותו.

מסמך זה מתאר את מצב הפרויקט נכון למועד הכתיבה ונועד להכנה לפגישת חיתוך המצב.